CI Workflow Guide

For Developers & Testers

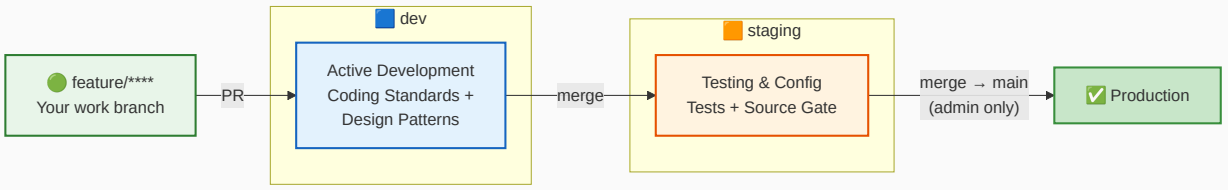
blurgsai/ci-workflows-software-team

github.com/blurgsai/ci-workflows-software-team

01

Branch Strategy

Four stages, one direction: **feature/*** → **dev** → **staging** → **main**.



Branch Hierarchy

feature/* → PR to dev created from staging created from main

Developers work on feature branches → PR to **dev**

Testers validate on **staging** → PR to **staging**

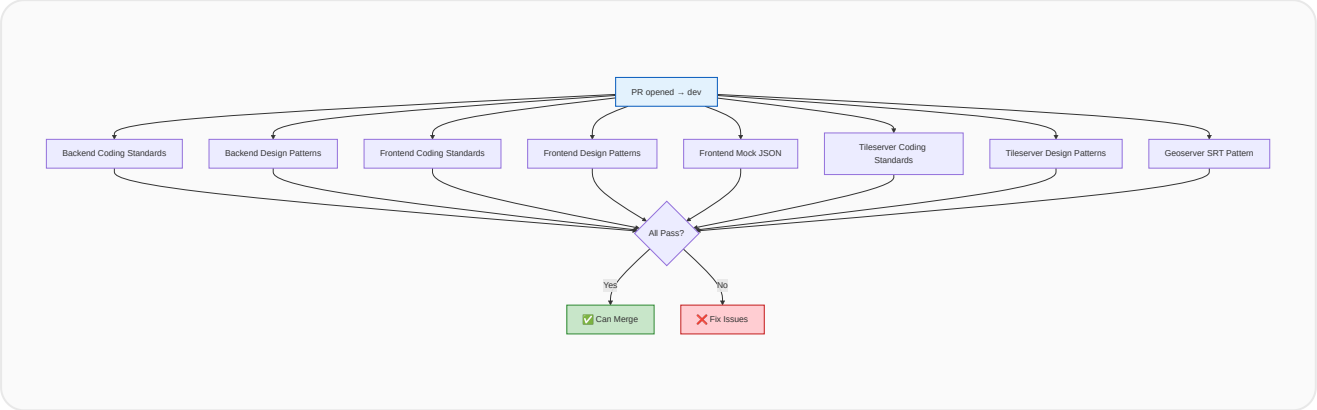
Only **staging** → **main** merges are allowed (via admin)

BRANCH	PURPOSE	CI RUNS	CI SKIPS
dev	Active development	Coding standards, design patterns, mock JSON, SRT	Tests
staging	Testing & config	Tests + source change rejection	Coding standards, design patterns
main	Production	❌ Always fails	Everything — direct merges blocked

02

Dev Branch CI

When a PR targets **dev**, all architecture and coding standard checks run. No tests.



What Gets Checked

- ✓ Naming conventions (files, functions, classes)
- ✓ Feature folder structure
- ✓ Layer import rules
- ✓ Barrel export rules
- ✓ Services independent of framework
- ✓ Router delegation rules
- ✓ Ruff / ESLint linters
- ✓ TypeScript type check
- ✓ Import-linter boundaries
- ✓ Mock JSON usage (frontend)
- ✓ SRT pattern (geoserver)

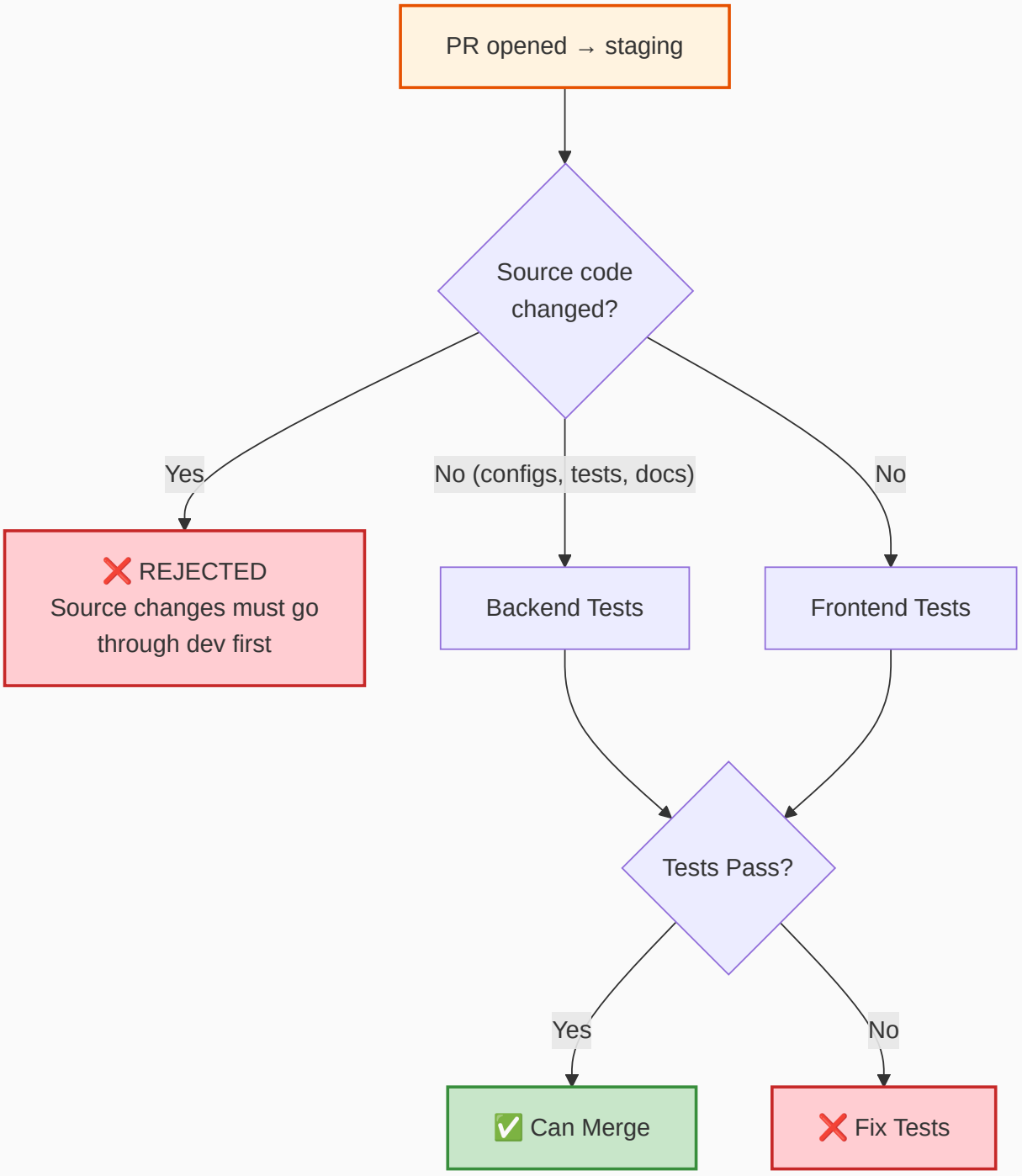
What Doesn't Run

- ✗ Backend unit tests
- ✗ Backend integration tests
- ✗ Frontend unit tests
- ✗ Frontend integration tests
- ✗ Test coverage checks
- ✗ Source code change rejection

03

Staging Branch CI

When a PR targets `staging`, a source code gate runs first. If it passes, tests run.



Source Code Gate — What is Rejected?

Any changes to **source code files** under `src/`:

- `backend/src/**/*.py`
- `tileserver/src/**/*.py`
- `frontend/src/**/*.ts`

- `frontend/src/**/*.tsx`
- `frontend/src/**/*.js`
- `frontend/src/**/*.jsx`

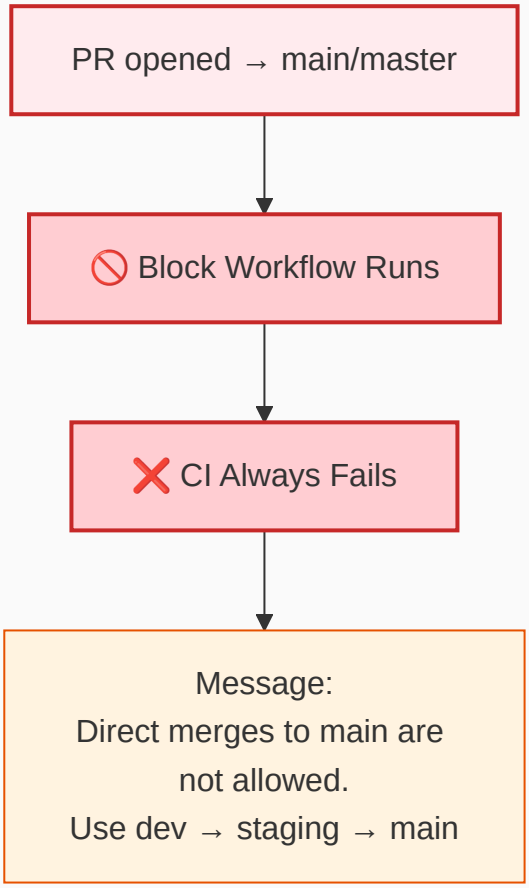
What IS Allowed on Staging?

- ✓ Test files (new tests, updated tests)
- ✓ Configuration files (pyproject.toml, .eslintrc, etc.)
- ✓ CI/CD files (.github/workflows/)
- ✓ Documentation (README, docs/)
- ✓ Infrastructure (docker-compose, scripts/)
- ✓ Dependencies (requirements.txt, package.json)

04

Main Branch Protection

Any PR or push to `main` or `master` will **always fail**.



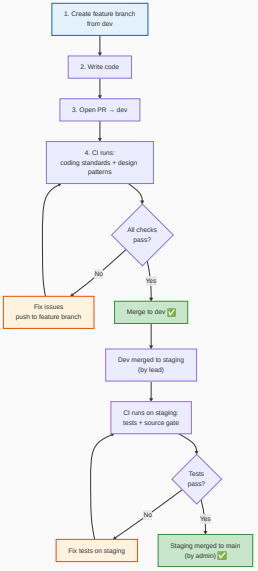
Why?

main should only receive code that has passed through **dev** (architecture checks) and **staging** (tests). This prevents unreviewed code from reaching production.

05

Developer Workflow

Step-by-step guide for developers pushing code changes.



1. **Branch off dev:** `git checkout dev && git pull && git checkout -b feature/your-feature`
2. **Write code** following naming conventions and feature folder structure
3. **Open PR to dev** — CI automatically runs all architecture checks
4. **Fix any failures** — push fixes to your feature branch, CI re-runs
5. **Merge to dev** once all checks pass ✅

06

Tester Workflow

Step-by-step guide for testers adding or updating tests.

1. **Branch off staging:** `git checkout staging && git pull && git checkout -b test/your-test`
2. **Write tests only** — do NOT modify any files under `src/`
3. **Open PR to staging** — source gate runs first, then tests
4. **If gate fails** — you accidentally changed source code. Move those changes to a `dev` PR.
5. **Merge to staging** once tests pass ✅

07

All CI Checks Overview

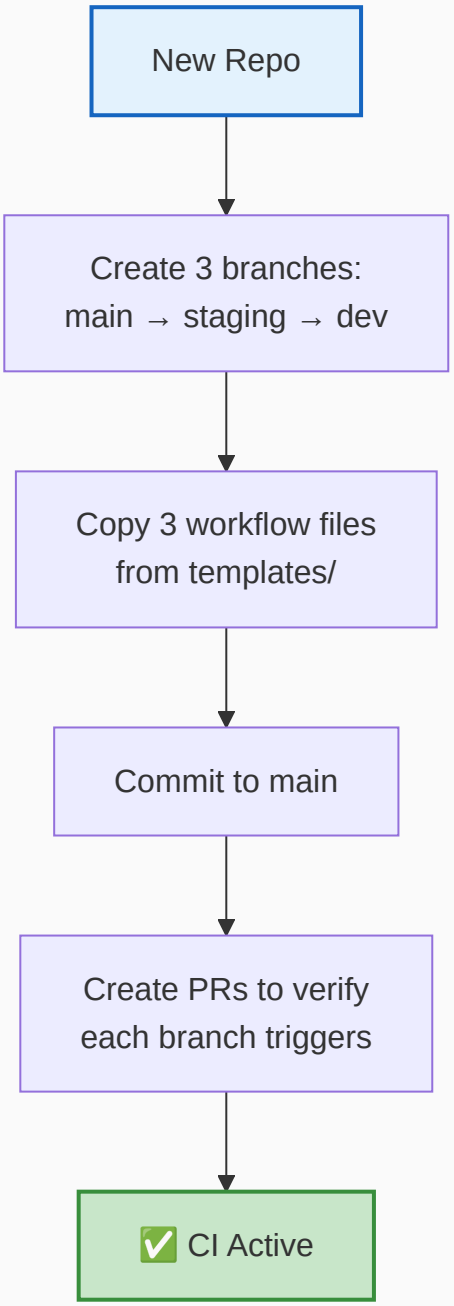
Every check that runs across all components and branches.

CHECK	COMPONENT	BRANCH	WHAT IT VALIDATES
Coding Standards	Backend	dev	snake_case files, PascalCase classes, Ruff linter
Design Patterns	Backend	dev	Feature folders, layer imports, barrel exports, router rules
Coding Standards	Frontend	dev	File naming, hook naming, ESLint, TypeScript check
Design Patterns	Frontend	dev	Feature folders, layer imports, hooks coverage, barrel exports
Mock JSON	Frontend	dev	Static JSON fetches, localStorage usage (warning)
Coding Standards	Tileserver	dev	snake_case files, Ruff linter
Design Patterns	Tileserver	dev	Feature folders, layer imports, router rules
SRT Pattern	Geoserver	dev	Docker compose, scripts, no hardcoded secrets
Tests	Backend	staging	Integration coverage, minimum count, pytest
Tests	Frontend	staging	Integration coverage, minimum count, vitest
Source Gate	All	staging	Rejects source code changes on staging
Block	All	main	Always fails — no direct merges

08

Repository Setup

How to add shared CI to a new repository.



Files to Add

Copy these from `templates/` in the shared CI repo to your repo's `.github/workflows/`:

- ✓ `dev-ci.yml` — triggers on PR/push to **dev**
- ✓ `staging-ci.yml` — triggers on PR/push to **staging**

✓ `block-main.yml` — triggers on PR/push to **main/master**

Branch Creation Order

1. `git checkout main && git checkout -b staging && git push -u origin staging`
2. `git checkout staging && git checkout -b dev && git push -u origin dev`

Auto-Skip Behavior

Workflows for **tileserver** and **geoserver** automatically skip if those directories don't exist in your repo. Same for **backend** and **frontend** — no configuration needed.